pyprims cheat sheet

Oct 17, 2026 · @Niels Taatgen

Setup

Install from the working copy with the notebook extras, then import the two entry points: `pyprims` for scripts and batch runs, `pyprims.notebook` for interactive work.

```
pip install -e ".[notebook,test]"     # in the pyprims working copy
python -m pytest tests                 # 53 tests
```

```
from pyprims import Model, Task, load_task, parse_prims, run_trials,
run_batch
from pyprims.notebook import Session, Dashboard, read_batch, plot_batch
```

Command line equivalents:

Command	Does
`python -m pyprims run count.prims -n 10 --seed 1 --trace 3`	Load all listed files, run the last one n trials, print the last trial's trace (level 0–5)
`python -m pyprims batch test.bprims -o out.dat --seed 1`	Run a .bprims batch script
`python -m pyprims convert model.prims -o outdir`	Convert .prims to the Python API (writes model.py)

Key terms

A `Session` contains one `Model`, and a `Model` can hold several `Task`s.

- **Task** (`Task`): what you define, in a .prims file, a .py file or Python code: its skills, operators, facts, actions, parameters and trial script.
- **Model** (`Model`): the running cognitive model: declarative memory, learned productions, buffers and time. Loading several tasks into one model gives transfer, because memory and productions are shared.
- **Session** (`Session`, notebooks only): wraps one model and adds stepping, inspection views, plots and the results of every run. `reset()` replaces the model with a fresh one; `s.model` is the current one.

- **Trial**: one run of a task's script; `run(n)` runs n trials of the current task.

Loading and running models

A `Model` loads a task from a .prims file, a .py file that defines `task`, or a `Task` object; loading a second task into the same model gives transfer.

In a notebook you don't create a `Model` yourself: a `Session` makes one (and a fresh one on each `reset()`), reachable as `s.model` (see Jupyter notebook functions below).

```
m = Model(seed=1)                    # trace=True
m.load("count.prims")                # or "count.py", or a Task object
m.run(50)                            # 50 trials of the current task
print(m.results[:5])                 # trial times (or plot-point values)
print(m.get_trace(3))                # last trial, levels 0–5
```

Model

Call	Does
`Model(seed=None, trace=True)`	New model. trace=False switches off the trace (faster runs); a seed makes runs reproducible
`m.load(task)`	Load a .prims/.py path or a `Task`; returns its task number
`m.switch_task(name)`	Make an already loaded task current (learning carries over)
`m.run(n=1)`	Run n trials of the current task
`m.run_trial()`	Run the task's script once
`m.set_parameter("rt", "-2.0")`	Set one parameter (PRIMs names, values as strings)
`m.results`	One value per trial: trial time, or the `plot-point` value
`m.output_data`	Batch events: actions, trial ends, data lines
`m.get_trace(level=3)`	Trace of the last trial as text
`m.dm.chunks`	All declarative chunks (dict by name)
`m.chunk("cf1").activation()`	One chunk and its activation
`m.operators_in_dm()`	The operator chunks

Call	Does
`m.production_table()`	Learned productions and their utilities

Defining a task in Python

A task in Python is one `Task` object. You create it, attach skills with their operators, facts, actions and a script to it, and then load it into a model. This is `examples/count.py`, shortened:

```
from pyprims import Task, Model

# 1. The task, with its parameters
task = Task("count", initial_skills=["count"], rt=-2.0, lf=0.2, ol=True)

# 2. A skill and its operators (bodies in PRIMs notation)
count = task.skill("count")
count.operator("start-count", """
    V1 <> nil
    WM1 = nil
==>
    V1 -> WM1
    count-fact -> RT1
    V1 -> RT2
    say -> AC1
    V1 -> AC2
""")
# ... the operators iterate and final in the same way

# 3. Facts and actions
task.facts([("cf1", "count-fact", "one", "two"),
            ("cf2", "count-fact", "two", "three")])
task.action("say", latency=0.3, noise=0.1, distribution="uniform")

# 4. The script: what happens in one trial of this task
@task.script
def script(m):
    m.screen("one", "three")
    m.run_until_action("say", "stop")
    m.issue_reward()
    m.trial_end()
```

```
# 5. Load it and run it
model = Model(seed=1)
model.load(task)          # in a notebook: s.load(task)
model.run(10)
```

The same file can be loaded by name (`model.load("count.py")`) as long as it defines a variable called `task`. In a notebook, a `%%pyprims` cell does steps 1–4 and loads the task in one go. The table lists the calls used above.

Call	Does
`Task("count", initial_skills=["count"], rt=-2.0, ol=True, ...)`	New task; parameters as keywords, `-` becomes `_`
`task.skill("count")`	Add a skill
`skill.operator("start-count", """V1 <> nil ==> V1 -> WM1 ...""")`	Add an operator; the body is PRIMs notation, unchanged
`task.facts([("cf1", "count-fact", "one", "two"), ...])`	Add facts to DM
`task.action("say", latency=0.3, noise=0.1, distribution="uniform")`	Define an action
`@task.script` on `def script(m): ...`	This task's trial script, run for each trial while this task is the current task; `m` is the script interface to the model (not the `Model` object) and has every script function
`load_task(path)` / `parse_prims(text)`	Build a Task from a file or from PRIMs text

Script functions are methods of `m` with `-` written as `_`. The common ones: `m.screen(...)`, `m.run_until_action("say", "stop")`, `m.run_relative_time_or_action(t, ...)`, `m.issue_reward()`, `m.trial_start()`, `m.trial_end()`, `m.plot_point(y)`, `m.data_line(...)`, `m.random(n)`, `m.shuffle(lst)`, `m.add_dm(...)`, `m.set_activation(...)`, `m.set_sji(...)`, `m.sgp(name, value)`, `m.set_goal(...)`, `m.instantiate_skill(...)`, `m.set_buffer_slot(...)`, `m.get_buffer_slot(...)`. The full list is in `tutorial/Reference.ipynb`.

Batch runs

Call	Does
`run_trials(m, n, task=None, events=False, **fields)`	Run n trials; returns one dict per trial end (`trial` , `task` , `result` , `outcome` , plus your fields)
`run_trials(..., events=True)`	One dict per event, with the batch-output columns
`run_batch("x.bprims", out_path=None, seed=None, progress=False)`	Run a .bprims file; returns the output lines

```
import pandas as pd
rows = []
for rep in range(10):
    m = Model(seed=rep, trace=False)
    rows += run_trials(m, 100, "count.prims", repeat=rep, label="train")
    rows += run_trials(m, 100, "semantic.prims", repeat=rep,
label="transfer")
data = pd.DataFrame(rows)
```

Jupyter notebook functions

In a notebook, wrap the model in a `Session` : it adds the Swift GUI's stepping, inspection panels and charts, and inspecting never changes the simulation.

```
s = Session(seed=1)            # trace_level=3, echo_print=False
s.load("count.prims")
s.step(); s.buffers(); s.conflict_set()
s.run(100)
s.plot_results(); s.plot_graph(2)
Dashboard(s)                   # the whole Swift window as widgets
```

Loading and controlling

Call	Does
`s.load(source)`	Load a task: a path to a .prims or .py file, a `Task` object, or model text in either syntax. A new task name adds the task to the model (transfer); a name already loaded redefines that task and resets the model
`s.load(source, name="other")`	The same, under another task name, e.g. to load one file as two separate tasks
`s.load_prims(text)` / `s.load_python(code)`	Load model text in one syntax
`s.select_task(name)`	Make a loaded task current
`s.reset()`	Fresh model with the current task; results are kept
`s.clear()`	Forget tasks, results and model
`s.step(n=1)`	Do n half steps (the Swift Step button); one by default
`s.step_operator(n=1)`	Step until n operators have executed; one by default
`s.finish_trial()`	Run the stepped trial to its end
`s.run(n=1)`	Run n trials; a stepped trial under way is finished first and counts as one. Only the last trial is traced
`s.run_time(seconds)`	Run trials until this much model time has passed
`s.stop()`	Stop a running `run` after the current trial
`s.batch("x.bprims")`	Run a batch script, independent of the session's model; returns the output lines
`s.batch("x.bprims", out_path="out.dat", seed=1)`	The same, also writing the lines to `out_path`; `seed` makes the batch reproducible
`s.model`	The underlying `Model`, for everything in the section above

Inspecting

Call	Shows
`s.status()`	Task, time, trial state, counts of operators, chunks, productions
`s.trace(level)` / `s.show_trace(level)`	Trace of the current or last trial, as text / as a display object. `level` 0–5: 0 script output only, 1 operators only, up to 5 everything; default the session's `trace_level` (3)
`s.buffers()`	Buffer panel
`s.conflict_set()`	Conflict trace: candidate operators and activation breakdown
`s.dm(type=None, contains=None)`	Declarative memory, sorted by activation. `type` keeps chunks of one type (e.g. `"operator"`); `contains` keeps chunks whose name contains that text
`s.chunk(name)`	One chunk: slots, activation breakdown, associations
`s.productions()`	Learned productions and utilities
`s.associations(source=None, type="operator", learned=False)`	Associations (Sji), strongest first. `source`: the skill (or other chunk) to show associations from, all by default; `type`: the kind of chunk they point to, operators by default, `None` for all; `learned=True`: only associations changed by learning
`s.tasks()`	Task list
`s.results` / `s.results_table()`	Trial results (list / DataFrame of task, run, trial, value)
`s.source(task=None)` / `s.to_python(task=None)`	A task's source text / the task converted to Python; `task` is a task name, the current task by default

Plotting

Call	Draws
`s.plot_results(average_window=None, title=None, tasks=None, labels=None)`	Results chart, one line per load or reset; repeated runs dashed
`s.plot_operator_activations(operators=None)`	Operator activations per step of the last trial; set `s.track_operator_activations = True` first
`s.plot_graph(level=2, seed=None, labels=True)`	The PRIMs graph
`read_batch(path_or_lines)`	Batch output as a DataFrame (.gz works)
`plot_batch(source, event="trial-end", p1="success", block=1, by=("task","label"))`	Mean event time per trial or block, ±1 SE over repeats
`figure(nrows, ncols, figsize)` / `show_figure(fig)`	Matplotlib helpers; plot functions take `ax=`

Cell magics

Importing `pyprims.notebook` registers two magics; each loads into a session called `session` unless you name another.

Magic	Does
`%%prims [name] [--reset] [--python]`	Cell in PRIMs syntax; `--python` also prints the Python version
`%%pyprims [name] [--reset]`	Cell that defines `task = Task(...)`; runs in the notebook namespace

Recipes and gotchas

- **Transfer:** load tasks one after another into the same model or session; DM and learned productions carry over. `m.load("count.prims"); m.run(50); m.load("semantic.prims"); m.run(20)`
- **Editing a model in a notebook:** re-run `s.load(...)` with the same task name; the task is redefined and the model reset.

- **Reproducibility:** `Session(seed=1)` draws a new model seed on each `reset()` from a seeded generator, so a whole sequence of actions repeats; `clear()` restarts that generator.
- **Failed trials:** after `s.run(n)`, a `NoOperatorWarning` reports trials where no operator matched or the trial timed out; set `s.warn_failed_trials = False` to silence it.
- **Fast batch runs:** use `Model(seed=rep, trace=False)`; tracing slows runs down.
- **Parameter names:** in `Task(...)` write `-` as `_`, `t` as `True`, `nil` as `False`; `m.set_parameter` takes the original PRIMs names as strings.
- **More:** `examples/pyprims_in_jupyter.ipynb` walks through the notebook interface; `tutorial/Reference.ipynb` lists every parameter and script function; `PORTING.md` lists the differences from Swift.